AIGC 5500 | WEEK 2

Optimization and Training Dynamics

Learning module

Week 1 got a network training. This week is about why that same network can fail to train at all once it gets deeper, and what four groups of tools — initialization, normalization, optimizers, and schedules — do about it. As with Week 1, you can work through this on your own or use it as your notes in class.

How to use this module

Four parts, each diagnosing a problem and then fixing it. Part 1 breaks a network on purpose; Parts 2 through 4 fix it in three different ways, and Lab 02 asks you to do the same to a network you have not seen before.

Part	Topic	Async time
Part 1	Why training breaks	30 minutes
Part 2	Initialization and normalization	45 minutes
Part 3	Modern optimizers	45 minutes
Part 4	Schedules, clipping, and mixed precision	40 minutes
Lab 02	Making training work	3 to 4 hours

The same three boxes from Week 1 appear throughout: green for an exercise to do now, blue for what happens in the live session, amber for a common mistake.

Part 1 · Why training breaks

In Week 1, the chain rule slide made one thing explicit: a gradient with respect to an early layer is a product of local derivatives, one per layer between that parameter and the loss. Everything in this part follows from taking that product seriously.

Vanishing gradients

Sigmoid and tanh squash their input into a narrow output range, and their local derivative is largest near zero and close to zero almost everywhere else. Stack enough layers using these activations, and the product of local derivatives from Week 1 shrinks multiplicatively with depth — by layer 20, a gradient that started at a reasonable size can be indistinguishable from zero. Early layers stop learning first, and stop learning worst, because they are furthest from the loss in the product.

Exploding gradients

The same product can run the other way. If local derivatives are consistently greater than 1 — common with poorly scaled weights — the product grows exponentially with depth instead of shrinking. The symptom is usually unmistakable: a loss that is finite for a few steps and then becomes NaN, because a weight update was large enough to push an activation somewhere the loss function cannot evaluate.

Dying ReLU

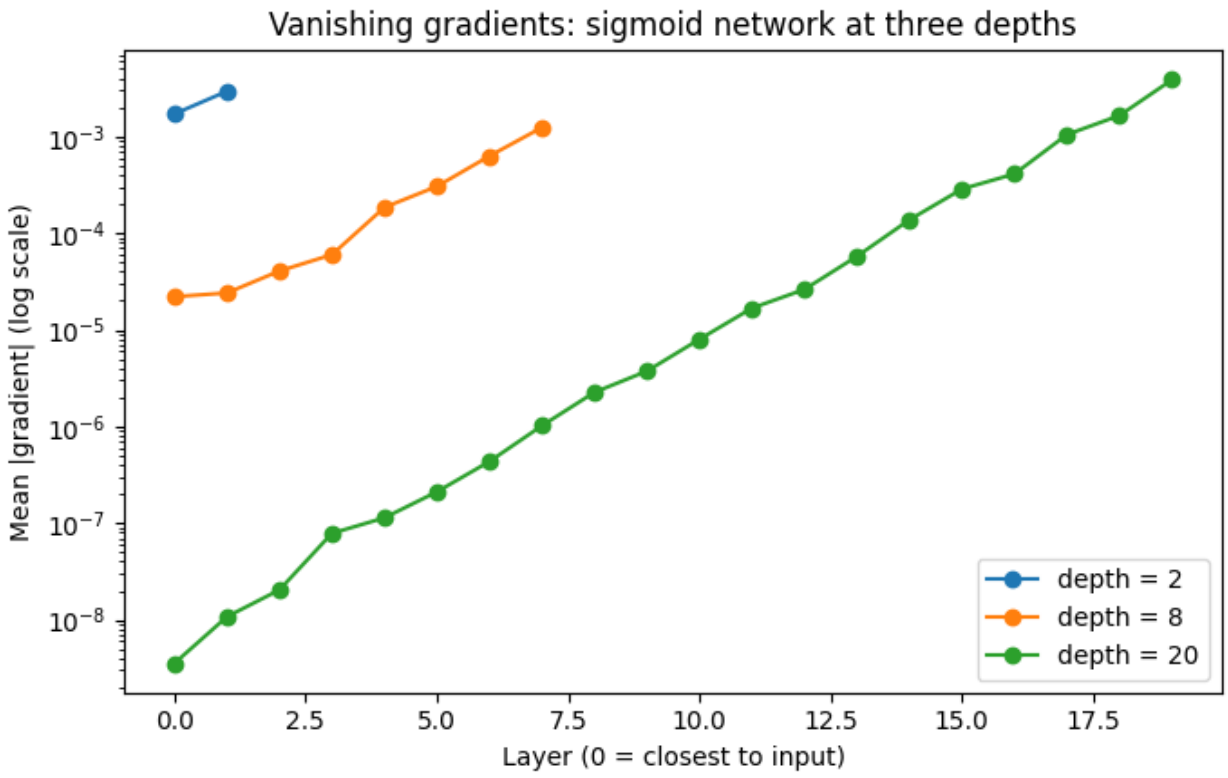
ReLU's local derivative is exactly 0 for any negative input and exactly 1 otherwise — no squashing, so no vanishing in the sigmoid sense. But a large enough negative update can push a ReLU unit's pre-activation negative for every input in your dataset. Once that happens, its gradient is 0 everywhere, no update can ever revive it, and that unit is permanently dead for the rest of training.

In the live session

We train the same small sigmoid network at three different depths (2, 8, and 20 layers) and watch the gradient norm at each layer collapse toward zero as depth increases, live.

Try it **Watch a gradient vanish**

1. Build a feedforward network with 10 hidden layers of size 32, each followed by a sigmoid activation.
2. Initialize all weights the same way (e.g. torch.nn.init.normal_ with a small fixed std) so the only variable is depth.
3. Run one forward and backward pass on a single batch, and print the mean absolute gradient of each layer's weights.
4. Confirm the gradient shrinks by roughly an order of magnitude every few layers going backward from the output.
5. Swap sigmoid for ReLU and re-run — the gradients should shrink far less, though ReLU brings its own failure mode from this part.



Part 2 · Initialization and normalization

If Part 1 is the diagnosis, this part is the first of three treatments. Both initialization and normalization attack the same root cause: activations whose scale drifts as they pass through layers, which is exactly what makes the chain-rule product from Part 1 shrink or explode in the first place.

Initialization: start at the right scale

If every layer roughly preserves the variance of its input, the product of local derivatives from Part 1 stays close to 1 per layer instead of drifting. Xavier (Glorot) initialization sets a layer's initial weight variance to 2 / (fan_in + fan_out), and was derived for sigmoid and tanh. Kaiming (He) initialization uses 2 / fan_in instead, accounting for the fact that ReLU zeroes out roughly half its inputs. Using Xavier init with ReLU networks, or He init with sigmoid networks, is a common and avoidable source of exactly the instability described in Part 1.

```
import torch.nn as nn

layer = nn.Linear(128, 128)
nn.init.kaiming_normal_(layer.weight, nonlinearity="relu")   # for ReLU networks
# nn.init.xavier_normal_(layer.weight)                       # for sigmoid/tanh
```

Normalization: keep it at the right scale

Initialization only fixes the starting scale; training itself can still drift. Batch normalization normalizes each layer's activations to zero mean and unit variance across the current mini-batch, then applies a learnable scale and shift so the network can undo the normalization if that turns out to help. This keeps activations — and therefore gradients — in a well-behaved range throughout training, not just at initialization, and it is also why batch-normalized networks tolerate much higher learning rates.

Layer normalization normalizes across a single example's features instead of across the batch, which makes it independent of batch size — the reason it is the default in Transformers (Week 5), where batch statistics are less meaningful. RMSNorm simplifies this further by skipping the mean-centering step and normalizing only by the root-mean-square, which is cheaper and is what most current large language models use.

```
self.norm1 = nn.BatchNorm1d(128)   # normalizes across the batch
self.norm2 = nn.LayerNorm(128)     # normalizes across features, per example

x = self.fc1(x)
x = self.norm1(x)
x = torch.relu(x)
```

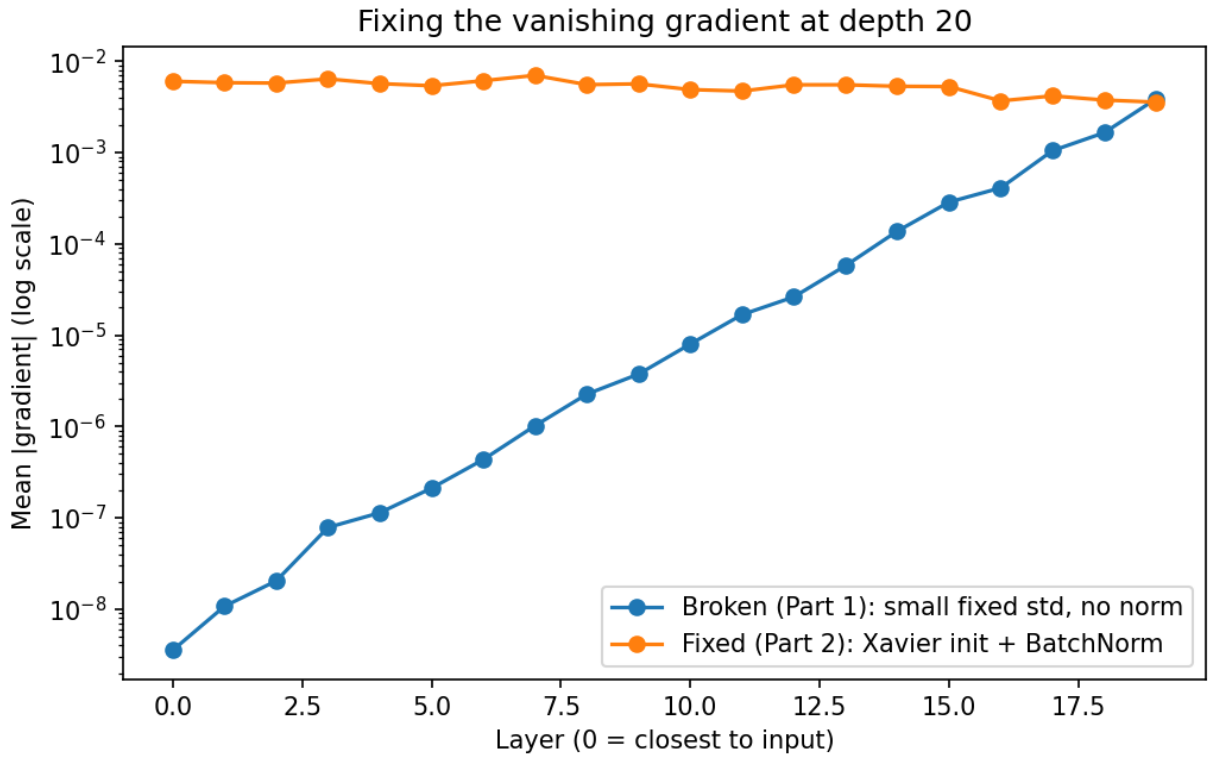
Watch out

BatchNorm computes statistics across the batch, so it behaves differently in train() and eval() mode — during evaluation it uses a running average collected during training rather than the current batch's statistics. Forgetting to call model.eval() before evaluating, or model.train() before resuming training, is one of the most common silent bugs involving BatchNorm, and it will not raise an error — it will just quietly give you the wrong numbers.

Try it **Fix the vanishing gradient from Part 1**

1. Take the 10-layer sigmoid network from Part 1's exercise.
2. Replace its default initialization with Xavier initialization on every layer.

3. Add a BatchNorm1d layer after each linear layer, before the sigmoid.
4. Re-run the same forward/backward pass and print the per-layer gradient norms again.
5. Confirm the gradients no longer shrink by an order of magnitude per layer the way they did in Part 1.



Part 3 · Modern optimizers

Initialization and normalization keep gradients well-scaled. This part is about what to do with a gradient once you have a good one — plain SGD is rarely the fastest or most stable way to turn a gradient into a parameter update.

Momentum: smoothing the path

Plain SGD updates a parameter using only the current step's gradient, which makes it sensitive to noisy or oscillating gradients — particularly in narrow ravines in the loss landscape, where it can bounce back and forth across the ravine instead of moving along it. Momentum keeps a running, exponentially-weighted average of past gradients (a velocity) and updates parameters using that average instead of the raw gradient, which damps oscillations and carries the optimizer through small local bumps and saddle points.

RMSprop: a learning rate per parameter

Different parameters can need very different effective learning rates — one weight might have consistently large gradients, another consistently small ones. RMSprop keeps a running average of each parameter's squared gradient and divides that parameter's update by the square root of this average, which shrinks the effective step size for parameters with large, noisy gradients and grows it for parameters with small, consistent ones.

Adam and AdamW

Adam combines both ideas: a running average of the gradient itself (like momentum) and a running average of the squared gradient (like RMSprop), each with a bias correction term that compensates for both averages starting at zero. It is a reasonable default for almost any architecture in this course. AdamW changes exactly one thing: it decouples weight decay from the gradient-based update rather than folding it into the gradient as a form of L2 regularization, which turns out to matter — Adam's adaptive per-parameter scaling interacts badly with weight decay folded into the gradient, weakening its regularizing effect in a way that is not obvious from the math alone. AdamW is the more defensible choice whenever weight decay is in use.

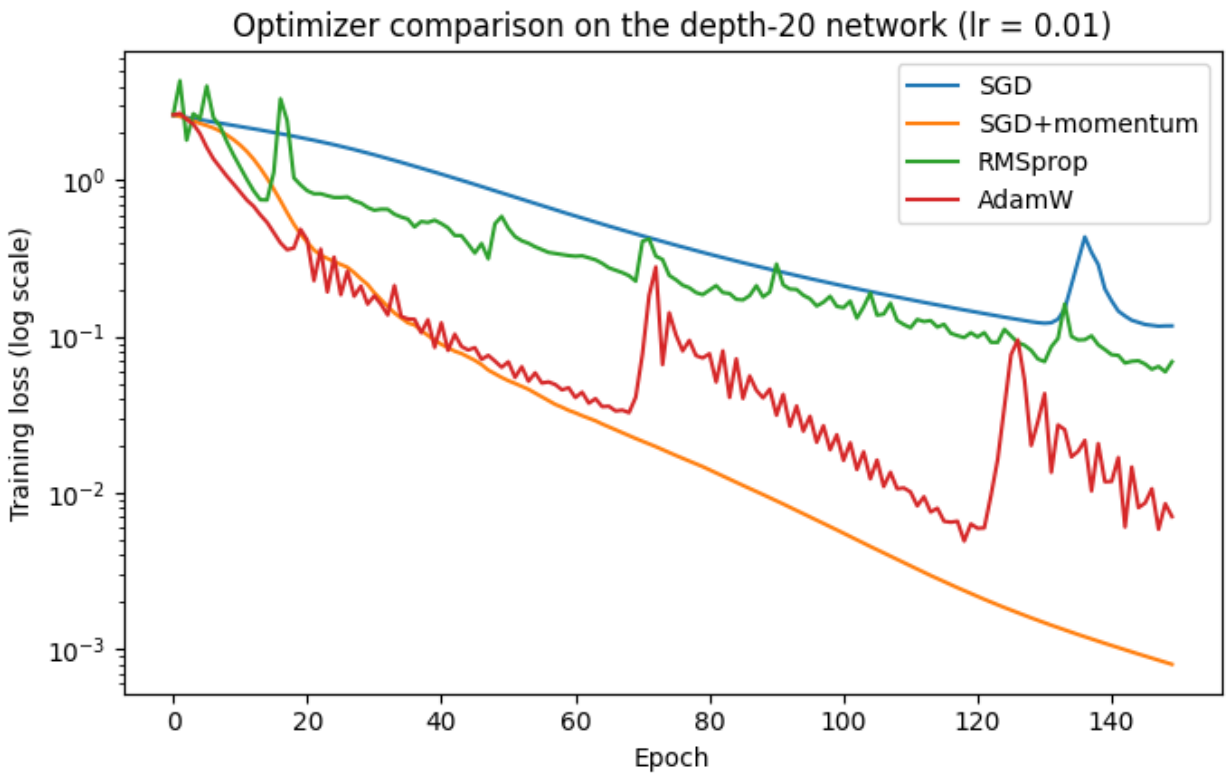
```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
# optimizer = torch.optim.RMSprop(model.parameters(), lr=0.001)
# optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
# optimizer = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
```

In the live session

We take the corrected network from Part 2 and train it four times — SGD, SGD with momentum, RMSprop, and AdamW — with everything else held fixed, and plot all four loss curves on the same axes.

Try it **Compare optimizers on the same network**

1. Using the corrected network from Part 2, train it once each with plain SGD, SGD with momentum=0.9, RMSprop, and AdamW.
2. Keep the learning rate, batch size, and number of epochs identical across all four runs.
3. Plot or print all four loss curves together.
4. Note which optimizer converges fastest, and whether any of them are unstable at this learning rate.



Part 4 · Schedules, clipping, and mixed precision

The last group of tools does not change how a gradient becomes an update — it changes when and how much of that update actually gets applied.

Learning rate schedules and warmup

A fixed learning rate is rarely optimal for an entire training run: a rate large enough to make fast progress early is often too large to fine-tune precisely late in training. Step decay drops the learning rate by a fixed factor at set epochs; cosine annealing decreases it smoothly following a cosine curve. Warmup does the opposite at the very start — it ramps the learning rate up from a small value over the first few hundred steps, which matters most with Adam-family optimizers, where the second-moment estimate from Part 3 is unreliable until it has seen enough gradients, making early updates noisier than they will be later.

```
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

for epoch in range(epochs):
    train_one_epoch(...)
    scheduler.step()
```

Gradient clipping

This is the direct fix for the exploding gradients described in Part 1: rescale the gradient if its norm exceeds a threshold, so a single bad batch cannot produce an update large enough to destabilize training. It costs almost nothing to add and almost never hurts, which is why it is common to see it in a training loop even when exploding gradients have not been observed yet.

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

Mixed precision and gradient accumulation

Mixed precision training runs most operations in float16 or bfloat16 instead of float32, which is faster and uses less memory on a GPU that supports it, while keeping a float32 copy of the weights for the update itself to avoid losing precision where it matters. Gradient accumulation solves a different, unrelated problem — a batch size too large to fit in memory — by running several smaller forward/backward passes and summing their gradients before calling optimizer.step() once, which is mathematically equivalent to training with the larger batch.

```
scaler = torch.cuda.amp.GradScaler()

with torch.autocast(device_type="cuda", dtype=torch.float16):
    preds = model(x)
    loss = criterion(preds, y)

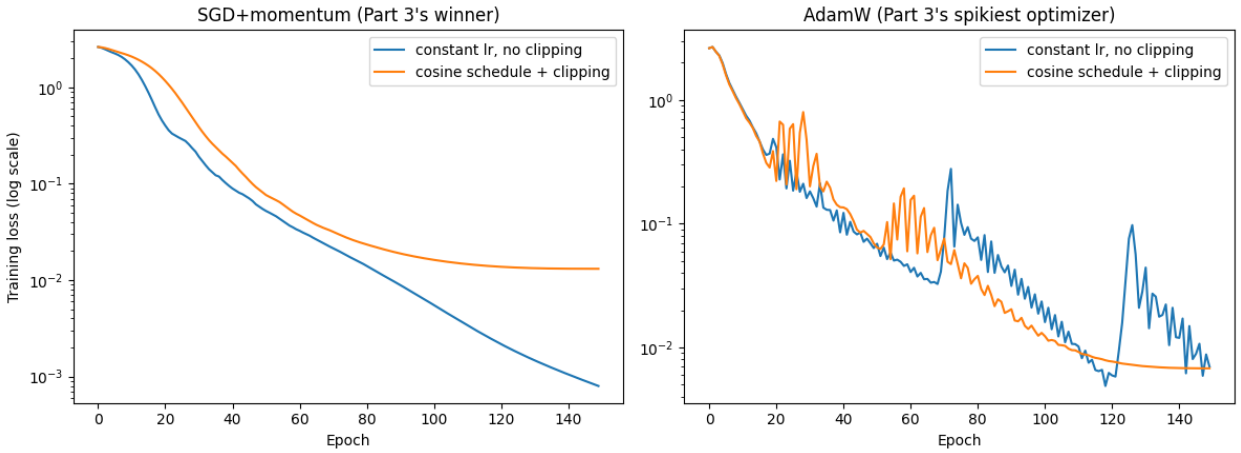
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

Watch out

Gradient clipping happens after loss.backward() and before optimizer.step() — it operates on the gradients that already exist in .grad, not on the loss or the weights directly. Calling it in the wrong position in the loop either does nothing or clips the wrong thing.

Try it **Add a schedule and clipping to your training loop**

1. Take your best-performing optimizer from Part 3's exercise.
2. Add a CosineAnnealingLR schedule across your training run.
3. Add gradient clipping with max_norm=1.0, placed correctly relative to backward() and step().
4. Re-run training and compare the loss curve to Part 3's unscheduled version, particularly in the last few epochs.



Check your understanding

Answer these without running code, then verify by running it. If you get one wrong, go back to the relevant part rather than moving on.

1. **Why do vanishing gradients hurt early layers more than late layers?**

   The gradient at a given layer is a product of local derivatives from that layer all the way to the loss. Early layers have more terms in that product between them and the loss, so if each local derivative is less than 1, early layers' gradients shrink the most.

2. **Why is a dying ReLU permanent, but a vanishing sigmoid gradient is not?**

   A vanishing sigmoid gradient is small but nonzero, so training still makes (very slow) progress. A ReLU unit whose pre-activation is negative for every input has a local gradient of exactly 0, so no update can ever change its weights again.

3. **What does batch normalization actually add to a layer's output, beyond normalizing it?**

   A learnable scale (gamma) and shift (beta), applied after normalizing to zero mean and unit variance. This lets the network undo the normalization if that turns out to help, rather than being permanently forced into a fixed distribution.

4. **Why does AdamW usually regularize better than Adam with weight decay folded into the gradient?**

   Adam scales each parameter's update by a running estimate of that parameter's gradient magnitude. When weight decay is folded into the gradient before this scaling, it gets scaled too, weakening its effect inconsistently across parameters. AdamW applies weight decay after Adam's scaling, so it acts uniformly as intended.

5. **What specific problem does warmup address that a lower learning rate alone does not?**

   Adam-family optimizers rely on a running estimate of the squared gradient, which is unreliable in the first few steps because it has not seen enough gradients yet. Warmup keeps the learning rate small while this estimate is still unreliable, then increases it once the estimate has stabilized — a fixed low learning rate throughout would not address that instability specifically.

6. **Where in the training loop does gradient clipping belong, and why?**

   After loss.backward() and before optimizer.step(), because it rescales the gradients already stored in each parameter's .grad — it has nothing to clip before backward() has populated .grad, and clipping after step() would be too late to affect the update.

When something goes wrong

These account for the large majority of Week 2 problems. Work through the table first.

Symptom	Most likely cause and fix
Loss becomes NaN partway through training	Almost always exploding gradients. Add gradient clipping (Part 4) first; if that does not resolve it, check your learning rate and initialization (Part 2) before anything else.
Loss barely moves from its initial value, at any depth	Check initialization first (wrong scheme for your activation function), then check whether your learning rate is simply too small.
Loss looks fine during training but validation accuracy is much worse	If you are using BatchNorm, confirm you called model.eval() before evaluating — BatchNorm behaves differently in train and eval mode, and this is the single most common cause of a train/eval mismatch.
Loss decreases nicely, then suddenly spikes and struggles to recover	A single bad batch produced a large update. Gradient clipping (Part 4) is the direct fix; a learning rate that is slightly too high is the usual underlying cause.
Two optimizers with the "same" learning rate behave completely differently	Expected. Adam-family optimizers and plain SGD are not directly comparable at the same nominal learning rate, since Adam rescales its updates per parameter. Tune each optimizer's learning rate separately rather than assuming one value transfers.
Mixed precision training produces NaN losses that full precision does not	float16 has a much smaller representable range than float32. Confirm GradScaler is actually being used (it rescales small gradients to avoid underflow) — this is the most common cause of mixed-precision-only instability.

Lab 02 and Week 3

Lab 02 gives you a network that will not train and asks you to fix it through a documented sequence of changes to initialization, normalization, optimizer, and schedule — the same four tools covered in this module's four parts, applied to a network you have not seen before. The diagnostic habits from Part 1 (checking gradient norms, watching for NaN losses) are what tell you which tool to reach for first.

Week 3 moves to convolutional networks, and assumes the training loop from here is reliable enough that you are not still debugging vanishing gradients while also learning a new architecture. If your network from this week's exercises is still unstable, resolve it before Week 3 rather than carrying the problem forward.